

El grounding en CogniTeam falla silenciosamente. Detecta que el plan no cumplió los requisitos pero el flujo continúa como "completado exitosamente". Vamos a diseñar una solución real.

El bug está en la arquitectura, no en el código. El Grounding Agent emite una recomendación ("Re-plan from scratch") pero el Orchestrator no la consume. Es como un semáforo en rojo que el conductor ignora.

Arquitectura corregida: Grounding como Gate, no como Advisor

Orchestrator → Planner → Developer → Grounding → Gate



└─ Score ≥ 0.7 → Continuar

└─ Score < 0.7 → Re-planificar (max 2 veces)



Score sigue < 0.7 → Abortar con error

El Grounding debe ser un **gate** que bloquea el flujo, no un **advisor** que sugiere y se ignora.

Implementación concreta

1. El `GroundingValidator` debe retornar un objeto de decisión, no un `string`

```
from pydantic import BaseModel
from typing import Literal

class GroundingResult(BaseModel):
    score: float # 0.0–1.0
    found_keywords: list[str]
    missing_keywords: list[str]
    verdict: Literal["pass", "fail", "partial"]
    max_attempts_reached: bool = False

    def should_continue(self) -> bool:
        return self.verdict == "pass" and self.score >= 0.7

    def should_replan(self) -> bool:
        return self.verdict in ("fail", "partial") and not self.max_attempts_reached

    def should_abort(self) -> bool:
        return self.verdict in ("fail", "partial") and self.max_attempts_reached

class GroundingValidator:
    def __init__(self, max_replans: int = 2):
        self.max_replans = max_replans

    def validate(
        self,
```

```
artifacts: dict,
expected_keywords: list[str],
attempt: int = 1,
) -> GroundingResult:
    # ... lógica actual de LLM ...

    score = self._calculate_score(artifacts, expected_keywords)

    return GroundingResult(
        score=score,
        found_keywords=found,
        missing_keywords=missing,
        verdict="pass" if score >= 0.7 else "fail",
        max_attempts_reached=(attempt > self.max_replans),
    )
```

2. El Orchestrator debe respetar el gate

```
class Orchestrator:
```

```
    def __init__(self):
```

```
        self.grounding = GroundingValidator(max_replans=2)
```

```
        self.replan_count = 0
```

```
    def run(self, task: str) -> dict:
```

```
        # 1. Scoping
```

```
        manifest = self.scoping.classify(task)
```

```
        # 2. Planificación (con posible re-planificación)
```

```
        plan = self._plan_with_grounding(manifest)
```

```
        # 3. Desarrollo
```

```
        artifacts = self._develop_with_grounding(plan, manifest)
```

```
        return artifacts
```

```
    def _plan_with_grounding(self, manifest: dict) -> list:
```

```
        """Genera plan y valida contra el manifiesto."""
```

```
        plan = self.planner.generate(manifest)
```

```
        # Grounding del plan (¿el plan cubre todos los requisitos?)
```

```
        grounding = self.grounding.validate(
```

```
            artifacts={"plan": plan},
```

```
            expected_keywords=manifest.get("required_technologies", []),
```

```
            attempt=1,
```

```
        )
```

```
        if grounding.should_replan():
```

```
            self.replan_count += 1
```

```

print(f" [Grounding] Re-planning (attempt {self.replan_count})...")

# Inyectar feedback al Planner

feedback = f"Missing: {grounding.missing_keywords}. Score: {grounding.score}"

plan = self.planner.generate(manifest, feedback=feedback)


# Re-validar

grounding = self.grounding.validate(
    artifacts={"plan": plan},
    expected_keywords=manifest.get("required_technologies", []),
    attempt=self.replan_count + 1,
)

if grounding.should_abort():
    raise GroundingAbortError(
        f"Grounding failed after {self.replan_count} attempts. "
        f"Missing: {grounding.missing_keywords}"
    )

return plan


def _develop_with_grounding(self, plan: list, manifest: dict) -> dict:
    """Ejecuta pasos y valida artefactos contra el plan."""
    artifacts = self.developer.execute(plan)

    # Grounding post-ejecución

    grounding = self.grounding.validate(
        artifacts=artifacts,
        expected_keywords=manifest.get("deliverables", []),
        attempt=1,
    )

```

```
if grounding.should_replan():

    # Injectar feedback al Developer

    feedback = f"Missing deliverables: {grounding.missing_keywords}"

    artifacts = self.developer.execute(plan, feedback=feedback)


if grounding.should_abort():

    raise GroundingAbortError(

        f"Post-execution grounding failed. "

        f"Missing: {grounding.missing_keywords}"

    )


return artifacts
```

3. TraceForge debe capturar el gate

```
@traceforge.trace(agent="grounding", model="gemma-4-31b")

def validate(self, artifacts, expected_keywords, attempt=1):

    result = self._llm_validate(artifacts, expected_keywords)


    # Guardar el score en el span para observabilidad

    span = traceforge.get_current_span()

    span.set_output({

        "score": result.score,

        "verdict": result.verdict,

        "attempt": attempt,

        "missing": result.missing_keywords,

    })


    return result
```

You are a Grounding Validator. Your job is to verify that the generated artifacts match the task requirements.

Rules:

1. Distinguish between TECHNOLOGIES (how it's built) and DELIVERABLES (what files exist).
2. A technology is "present" if the code uses it, not if a file with that name exists.
3. A deliverable is "present" if a file with the expected name and content exists.
4. WCAG 2.1 AA is a STANDARD, not a file. Check if the HTML has accessibility features (alt text, aria labels, keyboard navigation).

Output JSON:

```
{
  "score": 0.0-1.0,
  "verdict": "pass" | "partial" | "fail",
  "findings": [
    {"item": "Vanilla JS", "type": "technology", "present": true, "evidence": "script tags with inline JS"},
    {"item": "WCAG 2.1 AA", "type": "standard", "present": false, "evidence": "missing alt text on images"}
  ],
  "missing_but_optional": [],
  "missing_and_required": ["Astro Framework", "GSAP/Framer Motion"]
}
```


Métricas de salud del Grounding

TraceForge debería trackear:

Métrica	Qué indica	Umbral de alerta
grounding_score_avg	Calidad promedio del grounding	< 0.6 = problema sistémico
replan_rate	% de flujos que requieren re-planificación	> 30% = Planner débil
abort_rate	% de flujos que abortan por grounding	> 10% = Scoping insuficiente
replan_tokens_wasted	Tokens gastados en re-planificaciones	> 20% del total = ineficiente